Computer Architecture
Fall 2026
Project 1: Instruction Level Simulator
UPDATED

The initial work on the ISAs and the subsequent conversation in class led to a beautiful, natural decision in design philosophy. Both approaches are RISC-y in the context of modern ISAs. One approach favors fewer explicit opcode bits, and the other favors more opcode bits. Therefore… we are taking advantage of this providential turn of events! See below for the new and improved project instructions…

At the end of this assignment, each group will be turning in:

- A description of a well-defined ISA including a table with the following for each instruction in your ISA :
    - Instruction name
    - Instruction format (e.g. opcode | register | register. Something comparable to the R-type, I-type, J-type instruction formats given in MIPS)
    - RTL description of instruction (e.g. PC ← PC + 1, M(X) ← R(Y), etc.)
    - Description of instruction in words
    - For each instruction that is not obvious, include a note for when it is useful (e.g. ADD does not need an additional note. ShiftLeft might warrant a note. FP-Arith would definitely warrant a note.) Use your best judgement on which instructions need a note. (Tip: If any of the members of the group are unsure, it should have a note)
- An ISA level simulator as specified below
- A benchmark that uses every instruction in your ISA, the expected memory contents after the benchmark executes (this is for testing purposes)
- A MAXFINDER benchmark that will find the maximum of an array of integer numbers.
- A MAXFINDER benchmark that will find the maximum of an array of floating point numbers

Simulator Specifications

- You may write the simulator in any language as long as it can be run on onyx with no modifications of any sort to my environment.
- Your simulator will take in two arguments on the command line: a file containing an assembly language benchmark and a file specifying the memory contents at the beginning of execution.
- Your simulator will have two modes: 1) run straight through from beginning to end, 2) step-by-step. This will be specified on the command line by a flag: -r for run straight through, -s for step-by-step
- The usage statement will be: $ isaSim -<r|s> <assembly file> <memory file>
- You may define other flags if you so desire.

- Step-by-step Mode
    - At each step, display:
        - the current contents of all the registers mentioned in the RTL (e.g. PC, register file, etc.)
        - Total number of instructions executed so far
    - Wait for user to press enter before executing the next step (If you want to bonus it up, you may allow the user to enter a number of steps to execute before pausing and displaying the required data. That is not required, though)
    - At end of execution of the benchmark,
        - Display to the console the number of times each instruction was executed (e.g. how many times was `add` executed, how many times was `beq` executed, etc.)
        - Write the instruction count information out to a file called `<benchmarkName>.instrCount`
        - write the final contents of the memory to the appropriately named .out file(e.g. `<benchmarkName>.mem.out`)
- Run through Mode
    - At the end of execution of the benchmark,
        - Display to the console the number of times each instruction was executed (e.g. how many times was `add` executed, how many times was `beq` executed, etc.)
        - Write the instruction count information out to a file called `<benchmarkName>.instrCount`
        - write the final contents of the memory to the appropriately named .out file(e.g. `<benchmarkName>.mem.out`)

Memory File Format

The contents of the data memory will be given in a plain text file with a `.mem` extension the following format:

- First line: Comment with the name of the benchmark
- Second line: number of address bits, number of bits stored at each specified address (comma separated)
- All subsequent lines: Address in Hex, contents in hex \n (i.e. comma separates the address in Hex and the contents of that slot in memory in Hex)
- NOTE: Any address not specified will contain all 0's.
- NOTE: Assume the memory file is in the present working directory when the simulator is run.

Benchmark File Format

The benchmark to be executed will be written in assembly. In the process of writing your benchmark, it may be useful to use labels for loops or jumps, but when you execute your assembly program, replace any labels with concrete addresses. (e.g. if you have a branch-on-equal to LOOP1, replace that with the address where LOOP1 actually begins)

Submission Instructions

Due Sept 28 – Plan: Write a plain text document with the names of all team members, how you are are going to communicate (Email? Discord? Comments on the git repo? Smoke signals?), and how you are going to coordinate work. Submit to onyx using the following submit command:

```
submit sarahfrost compArchFA26 ISAplan
```

Due Oct 5 – ISA Specification: Submit your ISA specification document to Onyx using the following submit command:

```
submit sarahfrost compArchFA26 ISAspec
```

Due Oct 12 – Benchmarks: Each team will submit your benchmarks, memory files, and expected memory files to Onyx using the following submit command:

```
submit sarahfrost compArchFA26 ISAbench
```

Due Oct 19 – Simulator: Each team will submit their Instruction level simulator (raw documented code and all files needed to run the simulator) and a README in the format specified in modules to Onyx using the following submit command:

```
submit sarahfrost compArchFA26 ISASim
```

Grading

Plan 20 points

Assignment	Parts	Part pts	Total Points
Plan			20 pts
ISA Specification			150
	Descriptions + Notes	30 pts	
	RTL	40 pts	
	Format types (bits specified)	40 pts	
	Coherence of ISA	40 pts	
Benchmarks			200 pts
	Testing Benchmark	50 pts	

	MAXFINDER - integers	75 pts	
	MAXFINDER - floating point	75 pts	
Simulator			250
	Expected file formats, usage, programming conventions, etc. used	25	
	README	25	
	Step-by-step mode	100	
	Run through mode	100	